

PYTHON FOR QUANT TRADERS · PART III

Object-Oriented Programming

— สร้างระบบที่ขยายได้

บทที่ 18–21: Classes · Inheritance · Design Patterns · OOP Mini-Project

จบ Part นี้ → เปลี่ยน script ที่รันครั้งเดียวเป็น trading system ที่ใช้ได้จริง

ทำไมต้องเรียน **OOP**?

Code จาก Part II ทำงานได้ แต่มีปัญหาเมื่อโตขึ้น:

- เพิ่ม strategy ใหม่ → ต้องแก้ไขทุกที่มีโค้ดเก่า
- แשר code ระหว่าง strategy → copy-paste แล้วแก้
- เพิ่ม asset ใหม่ → เพิ่มตัวแปรกระจาย codebase

OOP แก้ปัญหาเหล่านี้ด้วยการ **จัดกลุ่มข้อมูลและ behavior** เข้าด้วยกันเป็น class — เปลี่ยนทีเดียว กระทบทุกที่

Running Example ตลอด **Part III — Refactor PairTradingSignal** จาก **Part II**

เราจะนำ PairTradingSignal จาก Part II มาออกแบบใหม่ให้เป็น OOP ที่ดี:

บท 18 → เข้าใจ class พื้นฐาน · บท 19 → Abstract base class + inheritance

บท 20 → ใส่ design patterns · บท 21 → Mini-project: ระบบครบวงจร

บทที่ 18: Classes & Objects

แก่นของบทนี้

Class คือ "blueprint" — นิยามว่า object ประเภทนี้เก็บข้อมูลอะไร (attributes) และทำอะไรได้บ้าง (methods) Object คือ "ชิ้นงานจริง" ที่สร้างจาก blueprint นั้น เหมือนแบบบ้าน (class) กับบ้านจริง (object)

18.1 ทำไมต้องมี Class — เรื่องเล่าจาก Pair ที่สอง

โค้ดสไตล์ Part I/II (ตัวแปร + ฟังก์ชัน) ทำงานได้ดีกับ pair เดียว:

```
# Part II style: ตัวแปรวางไว้ระดับบนสุดของไฟล์ window = 60 entry_z = 2.0 df =  
build_spread(btc_bybit, btc_binance, window) hl =  
calc_half-life(df["spread"]) signal = make_signal(df, entry_z)
```

ปัญหาเริ่มเมื่อเทรด **pair ที่สอง** ซึ่งใช้ parameter คนละชุด:

```
# เพิ่ม pair ที่สอง... ตัวแปรเริ่มแตกหน่อ window_2 = 90 # pair นี้ mean-revert ช้ากว่าให้  
window ยาวกว่า entry_z_2 = 1.5 df_2 = build_spread(eth_okx, eth_bybit,  
window_2) hl_2 = calc_half-life(df_2["spread"]) signal = make_signal(df,  
entry_z) signal_2 = make_signal(df_2, entry_z) # ← bug! ลืมเปลี่ยนเป็น entry_z_2  
# Python ไม่ฟ้องอะไรเลย — signal ผิดเจียบๆ
```

สังเกตว่า bug นี้เกิดเพราะข้อมูลกับ **parameter** ของแต่ละ **pair** ไม่ได้ผูกกัน — มันแค่ "วางอยู่ใกล้กัน" ในไฟล์ แล้วเราต้องจำเองว่าอะไรคู่กับอะไร ลองนึกต่อ: ถ้ามี **10 pairs** จะมีตัวแปร 40+ ตัวลอยอยู่ และทุกการเรียกฟังก์ชันคือโอกาสหยิบผิดคู่

Class คือคำตอบ: กล่องที่มีด "ข้อมูล + parameter + ฟังก์ชันที่ทำงานกับมัน" ไว้ด้วยกันเป็นชิ้นเดียว แล้วสร้างก็กล่องก็ได้:

```
# แต่ละ pair คือ object หนึ่งชิ้น — ถือของของตัวเองครบ pair1 = PairStrategy("BTC-Bybit",  
"BTC-Binance", window=60, entry_z=2.0) pair2 = PairStrategy("ETH-OKX",  
"ETH-Bybit", window=90, entry_z=1.5) signal1 = pair1.make_signal() # ใช้  
window=60, entry_z=2.0 ของตัวเองเสมอ signal2 = pair2.make_signal() # ใช้ของตัวเอง —  
หยิบสลับกันไม่ได้โดยโครงสร้าง
```

📌**อ่านว่า:** สั่งให้ `make_signal` ของ `pair1` ทำงาน — จุด (.) ยังอ่านว่า "ของ" เหมือนเดิม จาก Part I แค่นี้ "ของ" หมายถึงทั้งข้อมูลและฟังก์ชันที่อยู่ในกล่องเดียวกัน

CLASS ไม่ใช่ความรู้ใหม่

Class = **dict (เก็บข้อมูล) + function (ทำงาน)** เอามาผูกติดกัน — สองอย่างที่ใช้กล่องแล้วจาก Part I

ความใหม่มีแค่ 2 สิ่ง:

1. `self = "กล่องใบนี้"` — เมื่อ Python รัน `pair1.make_signal()` มันแปลงเป็น `PairStrategy.make_signal(pair1)` โดยอัตโนมัติ กล่าวคือ `self` คือ `pair1` ที่ถูกส่งเข้ามาเป็น argument แรกเสมอ — นี่คือ binding: function รู้ว่าตัวเองทำงานให้กล่องใบไหน
2. `self.window = "หยิบ window จาก dict ของกล่องใบนี้"` — ภายในเป็นแค่ dict ธรรมดา แค่ syntax ต่างกัน: `cfg["window"]` กับ `self.window` คือสิ่งเดียวกัน

18.2 Class พื้นฐาน

```
# แบบprocedural (Part I/II style) - ข้อมูลกระจาย symbol_a = "BTCUSDT-Bybit"
symbol_b = "BTCUSDT-Binance" window = 60 entry_z = 2.0 # แบบ OOP - ข้อมูลอยู่ในที่เดียว
class PairConfig: """เก็บconfiguration ของpair trading"""
    def __init__(self, symbol_a, symbol_b, window=60, entry_z=2.0, exit_z=0.5):
        self.symbol_a = symbol_a self.symbol_b = symbol_b self.window = window
        self.entry_z = entry_z self.exit_z = exit_z
    def __repr__(self): return (f"PairConfig({self.symbol_a} / {self.symbol_b}, "
                                f"window={self.window}, entry_z={self.entry_z})")
    def is_valid(self): """ตรวจสอบว่าconfig สมเหตุสมผล"""
        return (self.window >= 20 and self.entry_z > self.exit_z > 0)
# สร้าง object
cfg = PairConfig("BTCUSDT-Bybit", "BTCUSDT-Binance", window=60)
print(cfg) print(f"Valid: {cfg.is_valid()}")
```

► OUTPUT

```
PairConfig(BTCUSDT-Bybit / BTCUSDT-Binance, window=60, entry_z=±2.0)
Valid: True
```

18.3 ส่วนประกอบของ Class

ส่วนประกอบ	ใช้งาน	ตัวอย่าง
<code>__init__</code>	Constructor — รันเมื่อ <code>obj = Class(...)</code>	เก็บ parameters, สร้าง initial state
<code>__repr__</code>	String representation สำหรับ debug	<code>print(obj)</code> แสดงข้อมูลสำคัญ
<code>self</code>	ชี้ไปยัง object ปัจจุบัน	<code>self.window</code> คือ attribute ของ object นี้
Instance method	Function ที่รับ <code>self</code> เป็น argument แรก	<code>def is_valid(self):</code>
Class method	Method ที่แชร์ระหว่างทุก object	<code>@classmethod def from_dict(cls, d):</code>
Static method	Function ใน class ที่ไม่ใช้ <code>self</code>	<code>@staticmethod def annualize(daily):</code>

`self` กับ `cls` ต่างกันอย่างไร?

	<code>self</code>	<code>cls</code>
ชี้ไปที่	object ตัวนั้น (กล่องใบนี้)	class เอง (แบบพิมพ์เขียว)
ใช้ใน	method ทั่วไป	<code>@classmethod</code>
ตัวอย่าง	<code>self.window = window</code> ของ object นี้	<code>cls(rets, name) =</code> สร้าง object ใหม่จาก class

`@classmethod` ใช้บ่อยสำหรับ "alternative constructor" เช่น
`TradingMetrics.from_prices(prices)` = "สร้าง TradingMetrics จาก prices แทนที่จะส่ง returns โดยตรง"

```
class TradingMetrics: """คำนวณและเก็บ performance metrics"""
    TRADING_DAYS = 252
    # class variable — แชร์ทุก instance
    def __init__(self, returns, name="Strategy"):
        self.returns = np.array(returns)
        self.name = name
        self._cache = {}
        # _ prefix = "ควรใช้ภายใน class" (convention)
    @property
    def sharpe(self):
        """คำนวณ Sharpe Ratio (cached)"""
        if "sharpe" not in self._cache:
            m = self.returns.mean()
            s = self.returns.std()
            self._cache["sharpe"] = m / s * np.sqrt(self.TRADING_DAYS)
        return self._cache["sharpe"]
    @property
    def annual_return(self):
        return
```

```

self.returns.mean() * self.TRADING_DAYS @property def annual_vol(self):
return self.returns.std() * np.sqrt(self.TRADING_DAYS) @property def
max_drawdown(self): equity = np.cumprod(1 + self.returns) peak =
np.maximum.accumulate(equity) return ((equity - peak) / peak).min()
@classmethod def from_prices(cls, prices, name="Strategy"): """สร้าง
TradingMetrics จาก price series แทน returns""" rets = np.diff(prices) /
prices[:-1] return cls(rets, name) @staticmethod def annualize(daily_val,
periods=252): """Utility: annualize daily metric""" return daily_val *
np.sqrt(periods) def __repr__(self): return (f"TradingMetrics({self.name}):
" f"Sharpe={self.sharpe:.2f}, " f"Return={self.annual_return:.1%}, "
f"DD={self.max_drawdown:.1%}") # ใช้งาน import numpy as np
np.random.seed(42) rets = np.random.normal(0.0005, 0.012, 252) m =
TradingMetrics(rets, "BTC Pair") print(m) print(f"Annual Vol:
{m.annual_vol:.1%}")

```

► OUTPUT

```

TradingMetrics(BTC Pair: Sharpe=0.66, Return=12.6%, DD=-12.8%)
Annual Vol: 19.1%

```

18.4 @property — Attribute ที่คำนวณแบบ Lazy

@property ทำให้ method เรียกใช้ได้เหมือน attribute — ไม่ต้องใส่วงเล็บ และคำนวณเมื่อถูกเรียกเท่านั้น

```

# โดยไม่มี @property print(m.annual_vol()) # ต้องใส่ () - ดูแปลก # มี @property
print(m.annual_vol) # อ่านเหมือน attribute - ชัดเจนกว่า # ประโยชน์: lazy evaluation -
คำนวณเมื่อถูกเรียกเท่านั้น # ถ้า returns เปลี่ยน → ผลลัพธ์เปลี่ยนอัตโนมัติ

```

ตัวอย่าง: Position class พร้อม computed properties

```
class Position: """แทนตำแหน่งการเทรดเดียว""" def __init__(self, symbol, side, size, entry_price, fee_rate: float = 0.001): self.symbol = symbol self.side = side # "long" หรือ "short" self.size = size # จำนวน unit self.entry_price = entry_price self.exit_price = None self.fee_rate = fee_rate @property def is_open(self): return self.exit_price is None @property def pnl(self): if self.exit_price is None: raise ValueError("Position ยังไม่ปิด") raw = (self.exit_price - self.entry_price) * self.size return raw if self.side == "long" else -raw @property def return_pct(self): return self.pnl / (self.entry_price * self.size) def close(self, exit_price): self.exit_price = exit_price return self def __repr__(self): status = "OPEN" if self.is_open else f"CLOSED PnL={self.pnl:+,.0f}" return f"Position({self.side} {self.size} {self.symbol} @ {self.entry_price:,.0f} [{status}])" # ทดสอบ pos = Position("BTCUSDT", "long", 0.5, 63000) print(pos) pos.close(65500) print(pos) print(f"Return: {pos.return_pct:.2%}")
```

► OUTPUT

```
Position(long 0.5 BTCUSDT @ 63,000 [OPEN])
Position(long 0.5 BTCUSDT @ 63,000 [CLOSED PnL=+1,250])
Return: 3.97%
```

► แบบฝึกหัด: เพิ่ม fee calculation ให้ Position

```
class Position: def __init__(self, symbol, side, size, entry_price, fee_rate=0.001): # 0.1% maker fee self.symbol = symbol self.side = side self.size = size self.entry_price = entry_price self.exit_price = None self.fee_rate = fee_rate @property def gross_pnl(self): raw = (self.exit_price - self.entry_price) * self.size return raw if self.side == "long" else -raw @property def fees(self): entry_val = self.entry_price * self.size exit_val = self.exit_price * self.size return (entry_val + exit_val) * self.fee_rate @property def net_pnl(self): return self.gross_pnl - self.fees def close(self, exit_price): self.exit_price = exit_price return self
```

บทที่ 19: Inheritance & Abstract Base Class

แก่นของบทนี้

Inheritance ให้ class ลูกรับ attribute และ method จาก class แม่ —
ป้องกัน code ซ้ำ Abstract Base Class (ABC) กำหนด "สัญญา" ว่า class ลูก
ต้อง implement method ใดบ้าง เหมือน interface ใน Java/C++

19.1 Inheritance พื้นฐาน

```
class BaseStrategy: """Base class สำหรับทุก trading strategy""" def
__init__(self, name, capital=100_000): self.name = name self.capital =
capital self.trades = [] # history ของทุก trade self.equity = [capital] #
equity curve def log_trade(self, position: Position): """บันทึก trade ที่เปิด
แล้ว""" if position.is_open: raise ValueError("position นี้ยังไม่ถูกปิด - ต้อง close()
ก่อน") self.trades.append(position) self.capital += position.net_pnl
self.equity.append(self.capital) def summary(self): if not self.trades:
print(f"{self.name}: ยังไม่มี trade") return pnls = [t.net_pnl for t in
self.trades] winners = [p for p in pnls if p > 0] losers = [p for p in pnls
if p <= 0] print(f"=== {self.name} ===") print(f"Trades:
{len(self.trades)}") print(f"Win Rate: {len(winners)/len(pnls):.1%}")
print(f"Avg Win: {sum(winners)/len(winners) if winners else 0:+, .0f}")
print(f"Avg Loss: {sum(losers)/len(losers) if losers else 0:+, .0f}")
print(f"Total PnL: {sum(pnls):+, .0f}") print(f"Final Capital:
{self.capital:+, .0f}") def __repr__(self): return
f"{self.__class__.__name__}({self.name}, capital={self.capital:+, .0f})"
class PairTradingStrategy(BaseStrategy): """Pair trading ต่อยอดจาก
BaseStrategy""" def __init__(self, name, pair_config: PairConfig,
capital=100_000): super().__init__(name, capital) # เรียก __init__ ของแม่
self.config = pair_config self.signal_engine = None def fit(self, price_a,
price_b): """เทรน signal engine""" # PairTradingSignal ถูก import สมมติจาก Part
II - ในโปรเจกต์จริงให้วางในไฟล์เดียวกัน self.signal_engine = PairTradingSignal( window
= self.config.window, entry_z = self.config.entry_z, exit_z =
self.config.exit_z ).fit(price_a, price_b) return self def
get_signal(self): if self.signal_engine is None: raise RuntimeError("ต้อง
fit() ก่อน") return self.signal_engine.signal().iloc[-1] # ทดสอบ inheritance
cfg = PairConfig("BTC-Bybit", "BTC-Binance") strategy =
PairTradingStrategy("BTC Pair v1", cfg, capital=500_000) print(strategy)
print(f"เป็น BaseStrategy ไหม? {isinstance(strategy, BaseStrategy)}")
```


► OUTPUT

```
PairTradingStrategy(BTC Pair v1, capital=500,000)
เป็น BaseStrategy ไหม? True
```

ฝึกอ่าน: `super().__init__(name, capital)`

```
class PairTradingStrategy(BaseStrategy):
    def __init__(self, name, pair_config, capital=100_000):
        super().__init__(name, capital)  # ← บรรทัดนี้
```

อ่านจากในออกนอก:

1. `super()` — "คลาสแม่ของฉัน" = `BaseStrategy` (ที่อยู่ใน parentheses ของ class definition)
2. `super().__init__` — "เรียก `__init__` ของ คลาสแม่"
3. `super().__init__(name, capital)` — ส่ง `name` กับ `capital` ให้แม่ไปตั้งค่า `self.name`, `self.capital`, `self.trades`, `self.equity` เหมือนที่แม่ทำ

ถ้าไม่เรียก `super().__init__()` → `self.name` และ `self.capital` จะไม่ถูกสร้าง
→ เรียกทีหลังจะ `AttributeError` — ลูกต้องเรียกแม่ให้ทำงานของแม่ก่อน แล้วค่อยทำงานของตัวเอง

19.2 Abstract Base Class — บังคับให้ implement

```
from abc import ABC, abstractmethod
class Strategy(ABC):
    """Abstract base class – blueprint สำหรับทุก strategy"""
    def __init__(self, name, capital=100_000):
        self.name = name
        self.capital = capital
        self.trades = []
    @abstractmethod
    def fit(self, data):
        """ต้อง implement – train model"""
        pass
    @abstractmethod
    def signal(self, data) -> str:
        """ต้อง implement – คืน 'LONG', 'SHORT', 'EXIT', 'HOLD'"""
        pass
    @abstractmethod
    def position_size(self, signal: str) -> float:
        """ต้อง implement – คืน size เป็น fraction ของ capital"""
        pass
    def summary(self):
        # concrete method – ไม่ต้อง override
        pnls = [t.net_pnl for t in self.trades]
        total = sum(pnls) if pnls else 0
        print(f"{self.name}: {len(pnls)} trades, PnL={total:+.0f}")
        # ถ้าลืม implement abstractmethod จะ error
        # class BadStrategy(Strategy):
        #     pass
        # s = BadStrategy("test") ← TypeError: Can't instantiate abstract class
```

ฝึกอ่าน: `@abstractmethod` และ `ABC`

```
from abc import ABC, abstractmethod
```

```
class Strategy(ABC):          # ABC = Abstract Base Class
    @abstractmethod
    def signal(self, data) -> str:
        pass
```

อ่านทีละส่วน:

1. ABC ใน (ABC) — บอก Python ว่า "class นี้เป็น abstract — ห้ามสร้าง object โดยตรง"
2. @abstractmethod — decorator ที่ "ติดป้าย" ว่า method นี้ต้องถูก **override** โดยลูก — ถ้าลูกไม่ทำ สร้าง object ไม่ได้เลย
3. pass ใน method body — บอกว่า "แม้ไม่ได้ implement จริง ๆ แต่กำหนดชื่อ/ signature ไว้"

เปรียบ ABC กับ interface ของหมอ: "ทุกคนที่จบแพทย์ ต้อง มี license — ถ้าไม่มีจะทำงานเป็นหมอไม่ได้" — ABC คือการบังคับแบบเดียวกัน: ทุก strategy ต้อง implement `fit()` และ `signal()`

ประโยชน์ของ ABC ใน trading system

- บังคับว่าทุก strategy ต้องมี `fit()`, `signal()`, `position_size()`
- Backtester ทำงานกับ Strategy ประเภทไหนก็ได้ — ไม่ต้องรู้ว่าเป็น pair, momentum, หรือ vol
- เพิ่ม strategy ใหม่ → implement 3 methods → ทำงานกับ backtester ทันที

19.3 Encapsulation — ซ่อน implementation detail

```
class RiskManager: """จัดการ risk - ซ่อน implementation ภายใน"""
    def __init__(self, max_position_pct=0.20, max_drawdown_pct=0.10):
        self._max_pos = max_position_pct # _ = "private by convention"
        self._max_dd = max_drawdown_pct
        self.__peak_equity = None # __ = name-mangled (private จริงๆ)
    def check(self, signal, capital, current_equity):
        """คืน True ถ้า trade ผ่าน risk check"""
        if signal == "HOLD":
            return False # ถ้าเจอ peak
        if self.__peak_equity is None or current_equity > self.__peak_equity:
            self.__peak_equity = current_equity # Drawdown check
            drawdown = (current_equity - self.__peak_equity) / self.__peak_equity
            if drawdown < -self._max_dd:
                print(f"RISK BLOCK: Drawdown {drawdown:.1%} เกิน limit {-self._max_dd:.1%}")
                return False
            return True
    def size(self, signal, capital, volatility):
        """คำนวณ position size ที่ปลอดภัย"""
        if volatility <= 0:
            return 0 # Vol-adjusted sizing: target 1% daily vol contribution
        target_vol = 0.01
        raw_size = target_vol / volatility
        return min(raw_size, self._max_pos) # ไม่เกิน max position
    @property
    def max_position(self):
        return self._max_pos # max_position อ่านได้ แต่เปลี่ยนตรงๆ ไม่ควร rm =
```

```
RiskManager(max_position_pct=0.15, max_drawdown_pct=0.10) print(f"Max position: {rm.max_position:.0%}") print(f"Approved: {rm.check('LONG', 500000, 490000)}")
```

ตัวอย่าง: PairStrategy ที่ implement ABC ครบ

```
# ต้องimport: from statsmodels.regression.rolling import RollingOLS
class PairStrategy(Strategy): """Concrete pair trading strategy –
implement ครบ 3 abstract methods""" def __init__(self, name,
symbol_a, symbol_b, window=60, entry_z=2.0, capital=100_000):
super().__init__(name, capital) self.symbol_a = symbol_a
self.symbol_b = symbol_b self.window = window self.entry_z = entry_z
self._beta = None self._zscore = None def fit(self, data:
pd.DataFrame): """data: DataFrame ที่มี column symbol_a และ symbol_b"""
log_a = np.log(data[self.symbol_a]) log_b =
np.log(data[self.symbol_b]) X = sm.add_constant(log_b) roll =
RollingOLS(log_a, X, window=self.window).fit() self._beta =
roll.params.iloc[:, 1] alpha = roll.params['const'] spread = log_a -
(alpha + self._beta * log_b) roll_m =
spread.rolling(self.window).mean() roll_s =
spread.rolling(self.window).std() self._zscore = (spread - roll_m) /
roll_s return self def signal(self, data=None) -> str: z =
self._zscore.iloc[-1] if z > self.entry_z: return "SHORT" if z < -
self.entry_z: return "LONG" if abs(z) < 0.5: return "EXIT" return
"HOLD" def position_size(self, signal: str) -> float: if signal ==
"HOLD": return 0.0 if signal == "EXIT": return 0.0 return 0.10 # 10%
ของ capital (จะ optimize ใน Part IV) # ตรวจสอบว่าimplement ครบ strat =
PairStrategy("BTC Pair", "Bybit", "Binance") print(isinstance(strat,
Strategy)) # True print(strat)
```

► แบบฝึกหัด: สร้าง **MomentumStrategy** ที่ implement ABC

```
class MomentumStrategy(Strategy): """Momentum strategy: long เมื่อ MA20 >
MA50""" def __init__(self, name, symbol, fast=20, slow=50,
capital=100_000): super().__init__(name, capital) self.symbol = symbol
self.fast = fast self.slow = slow self._ma_fast = None self._ma_slow =
None def fit(self, data: pd.DataFrame): prices = data[self.symbol]
self._ma_fast = prices.rolling(self.fast).mean() self._ma_slow =
prices.rolling(self.slow).mean() return self def signal(self,
data=None) -> str: if self._ma_fast.iloc[-1] > self._ma_slow.iloc[-1]:
return "LONG" elif self._ma_fast.iloc[-1] < self._ma_slow.iloc[-1]:
return "SHORT" return "HOLD" def position_size(self, signal: str) ->
float: return 0.20 if signal != "HOLD" else 0.0
```

บทที่ 20: Design Patterns

แก่นของบทนี้

Design Patterns คือ "วิธีแก้ปัญหาซ้ำๆ ที่ผ่านการพิสูจน์แล้ว" ใน Quant system ใช้บ่อย 3 patterns: **Strategy Pattern** (สลับ algorithm ได้โดยไม่แก้ backtester), **Observer Pattern** (แจ้งเตือนเมื่อมี event เช่น signal เกิด), **Factory Pattern** (สร้าง object จาก config โดยไม่ hard-code ชื่อ class)

20.1 Strategy Pattern — สลับ Algorithm

Backtester ทำงานกับ ทุก strategy โดยไม่รู้ว่าเป็นประเภทไหน — ขอแค่ implement Strategy ABC

```
class Backtester: """ Generic backtester - ทำงานกับ Strategy ใดก็ได้ Strategy
Pattern: algorithm (strategy) แยกจาก runner (backtester) """
    def __init__(self, strategy: Strategy, risk_manager: RiskManager = None):
        self.strategy = strategy
        self.rm = risk_manager or RiskManager()
        self.results = []
    def run(self, data: pd.DataFrame, price_col: str) -> pd.DataFrame:
        """ รัน backtest บน data data: DataFrame ที่มีราคา price_col: ชื่อ
        column ของราคาที่ใช้ execute trade """
        prices = data[price_col].values
        capital = self.strategy.capital
        equity = [capital]
        pos = None # position ปัจจุบัน
        self.strategy.fit(data.iloc[:60]) # train บน 60 วันแรก
        for i in range(60, len(data)):
            window_data = data.iloc[max(0, i-60):i]
            sig = self.strategy.signal(window_data)
            price = prices[i] # Exit logic
            if pos and (sig == "EXIT" or (sig == "LONG" and pos.side == "short") or (sig == "SHORT" and pos.side == "long")):
                pos.close(price)
                capital += pos.net_pnl
                self.strategy.trades.append(pos)
                pos = None # Entry logic
            if pos is None and sig in ("LONG", "SHORT"):
                if self.rm.check(sig, capital, capital):
                    frac = self.strategy.position_size(sig)
                    size = (capital * frac) / price
                    side = "long" if sig == "LONG" else "short"
                    pos = Position(price_col, side, size, price)
                    equity.append(capital)
                    data = data.copy()
                    data["equity"] = equity[:len(data)]
        return data # ใช้งาน Strategy Pattern: # สลับระหว่าง pair กับ
        momentum โดยไม่แก้ Backtester เลย
        pair_bt = Backtester(PairStrategy("BTC Pair", "Bybit", "Binance"))
        mom_bt = Backtester(MomentumStrategy("BTC Mom", "Bybit"))
```

20.2 Observer Pattern — Event System

Observer ให้ object "สมัครรับ" event จาก object อื่น — เมื่อ event เกิด ทุก observer ถูกแจ้ง

```
from typing import Callable, List class EventBus: """ Simple event bus –
Observer Pattern ให้สำหรับ: signal เกิด → แจ้ง logger, risk manager, order
executor """ def __init__(self): self._listeners: dict[str, List[Callable]]
= {} def subscribe(self, event: str, callback: Callable): """สมัครรับ event"""
self._listeners.setdefault(event, []).append(callback) def publish(self,
event: str, data=None): """ส่ง event ให้ทุก subscriber""" for cb in
self._listeners.get(event, []): cb(data) # ตัวอย่าง: เมื่อ signal เกิด → logger,
risk, executor ทำงาน bus = EventBus() # Subscribers def
on_signal_logger(data): print(f"[LOG] Signal: {data['signal']} @
{data['price']:, .0f}") def on_signal_risk(data): if data['signal'] !=
"HOLD": print(f"[RISK] Checking signal: {data['signal']}") def
on_signal_execute(data): if data['signal'] == "LONG": print(f"[EXEC]
Placing LONG order @ {data['price']:, .0f}") bus.subscribe("signal",
on_signal_logger) bus.subscribe("signal", on_signal_risk)
bus.subscribe("signal", on_signal_execute) # Publish event
bus.publish("signal", {"signal": "LONG", "price": 65000})
```

► OUTPUT

```
[LOG] Signal: LONG @ 65,000
[RISK] Checking signal: LONG
[EXEC] Placing LONG order @ 65,000
```

20.3 Factory Pattern — สร้าง Object จาก Config

```
class StrategyFactory: """ Factory Pattern: สร้าง strategy object จาก config
dict ป้องกัน hard-code ชื่อ class ใน code หลัก """ _registry = {} @classmethod def
register(cls, name: str, strategy_class): cls._registry[name] =
strategy_class @classmethod def create(cls, config: dict) -> Strategy:
strat_type = config.get("type") if strat_type not in cls._registry: raise
ValueError(f"Unknown strategy: {strat_type}. " f"Available:
{list(cls._registry.keys())}") klass = cls._registry[strat_type] return
klass(**{k: v for k, v in config.items() if k != "type"}) # ลงทะเบียน
strategies StrategyFactory.register("pair", PairStrategy)
StrategyFactory.register("momentum", MomentumStrategy) # สร้างจาก config (อาจมา
จาก JSON file) config = { "type": "pair", "name": "BTC Pair v2", "symbol_a":
"Bybit", "symbol_b": "Binance", "window": 60, "entry_z": 2.0, "capital":
500_000 } strat = StrategyFactory.create(config) print(strat)
print(type(strat).__name__)
```

► OUTPUT

```
PairStrategy(BTC Pair v2, capital=500,000)
PairStrategy
```

เมื่อไหร่ควรใช้ **Design Pattern** แต่ละแบบ

Pattern	ใช้เมื่อ	ตัวอย่างใน trading
Strategy	มีหลาย algorithm ที่สลับกันได้	Pair / Momentum / Volatility strategy
Observer	หลาย component ต้องรู้เมื่อ event เกิด	Signal → Log + Risk + Execute
Factory	สร้าง object จาก config โดยไม่ hard-code	Load strategy จาก JSON config

บทที่ 21: OOP Mini-Project — Trading System

แก่นของบทนี้

รวมทุกอย่างจาก Part III เป็น trading system ที่ใช้งานได้จริง: **Portfolio** (เก็บ positions หลายตัว), **RiskManager** (ควบคุม risk), **Backtester** (ทดสอบย้อนหลัง), **StrategyFactory** (สร้างจาก config) — ทั้งหมดเชื่อมกันด้วย EventBus

21.1 Architecture Overview



21.2 Portfolio class

```
class Portfolio: """ จัดการ positions หลายตัวพร้อมกัน เก็บ state ทั้งหมดของระบบการเทรด """
def __init__(self, initial_capital=1_000_000): self.initial_capital = initial_capital
self.cash = initial_capital
self.open_positions = {} # symbol -> Position
self.closed_positions = []
self._equity_history = [initial_capital]
def open(self, symbol, side, size, price, fee_rate=0.001):
    """เปิด position ใหม่"""
    if symbol in self.open_positions:
        raise ValueError(f"มี position {symbol} อยู่แล้ว")
    cost = size * price * (1 + fee_rate)
    if cost > self.cash:
        raise ValueError(f"เงินไม่พอ: ต้องการ {cost:,.0f}, มี {self.cash:,.0f}")
    self.cash -= cost
    self.open_positions[symbol] = Position(symbol, side, size, price, fee_rate)
    return self
def close(self, symbol, price):
    """ปิด position"""
    if symbol not in self.open_positions:
        raise ValueError(f"ไม่มี position {symbol}")
    pos = self.open_positions.pop(symbol)
    pos.close(price)
    self.cash += pos.size * price * (1 - pos.fee_rate)
    self.closed_positions.append(pos)
    self._equity_history.append(self.total_equity({symbol: price}))
    return pos
def total_equity(self, current_prices: dict) -> float:
    """มูลค่ารวม = cash + unrealized positions"""
    unrealized = sum( pos.size * current_prices.get(sym, pos.entry_price) for sym, pos in
```



```

self.open_positions.items() ) return self.cash + unrealized @property def
metrics(self): pnls = [p.net_pnl for p in self.closed_positions] if not
pnls: return {"trades": 0} winners = [p for p in pnls if p > 0] return
{ "trades": len(pnls), "win_rate": len(winners) / len(pnls), "total_pnl":
sum(pnls), "avg_win": sum(winners)/len(winners) if winners else 0,
"avg_loss": sum(p for p in pnls if p <= 0) / max(1, len(pnls)-
len(winners)) } def __repr__(self): return (f"Portfolio(cash={self.cash:,.
0f}, " f"positions={list(self.open_positions.keys())}, "
f"trades={len(self.closed_positions)}")

```

21.3 ประกอบทุก Component เป็น TradingSystem

```

class TradingSystem: """ Facade class – จัดการทุกอย่างผ่านจุดเดียวไว้: system =
TradingSystem.from_config(config) """ def __init__(self, strategy:
Strategy, portfolio: Portfolio, risk_manager: RiskManager): self.strategy =
strategy self.portfolio = portfolio self.rm = risk_manager self.bus =
EventBus() self._setup_listeners() def _setup_listeners(self):
self.bus.subscribe("signal", self._on_signal) self.bus.subscribe("error",
self._on_error) def _on_signal(self, data): sig = data["signal"] price =
data["price"] sym = data["symbol"] if sig == "LONG" and sym not in
self.portfolio.open_positions: capital = self.portfolio.total_equity({}) if
self.rm.check(sig, capital, capital): frac =
self.strategy.position_size(sig) size = (capital * frac) / price
self.portfolio.open(sym, "long", size, price) print(f"[OPEN] LONG {sym}
size={size:.4f} @ {price:,.0f}") elif sig in ("EXIT", "SHORT") and sym in
self.portfolio.open_positions: pos = self.portfolio.close(sym, price)
print(f"[CLOSE] {sym} PnL={pos.net_pnl:+,.0f}") def _on_error(self, data):
print(f"[ERROR] {data}") @classmethod def from_config(cls, config: dict):
"""สร้างระบบทั้งหมดจาก config dict""" strategy =
StrategyFactory.create(config["strategy"]) portfolio =
Portfolio(config.get("capital", 100_000)) risk_mgr =
RiskManager( max_position_pct=config.get("max_position", 0.20),
max_drawdown_pct=config.get("max_drawdown", 0.10) ) return cls(strategy,
portfolio, risk_mgr) def step(self, symbol, price, data): """ประมวลผล 1
timestep""" sig = self.strategy.signal(data) self.bus.publish("signal",
{"signal": sig, "price": price, "symbol": symbol}) def report(self): m =
self.portfolio.metrics print(f"\n=== Final Report: {self.strategy.name}
===") for k, v in m.items(): if isinstance(v, float): print(f" {k:12s}:
{v:.2%}" if "rate" in k else f" {k:12s}: {v:+,.0f}") else: print(f" {k:
12s}: {v}")

```

21.4 ทดลองใช้ระบบทั้งหมด

```

# Config ทั้งระบบในที่เดียว SYSTEM_CONFIG = { "capital": 1_000_000,
"max_position": 0.15, "max_drawdown": 0.10, "strategy": { "type": "pair",
"name": "BTC Pair Production", "symbol_a": "Bybit", "symbol_b": "Binance",
>window": 60, "entry_z": 2.0, "capital": 1_000_000 } } # สร้างระบบ system =
TradingSystem.from_config(SYSTEM_CONFIG) # จำลองข้อมูลและวิ่ง import numpy as np,

```

```

pandas as pd np.random.seed(0) n = 120 dates = pd.date_range("2024-01-01",
periods=n) bybit = 65000 + np.cumsum(np.random.normal(0, 80, n)) binance =
bybit + np.random.normal(0, 20, n) # ติดกับ bybit df = pd.DataFrame({"Bybit":
bybit, "Binance": binance}, index=dates) # Train
system.strategy.fit(df.iloc[:60]) # Live simulation (60 steps) print("===
Simulating 60 days ===") for i in range(60, n): window =
df.iloc[max(0,i-60):i] price = df["Bybit"].iloc[i] system.step("Bybit",
price, window) system.report()

```

จบ Part III — สิ่งที่ได้จาก OOP

ก่อน **OOP (Part II)**: functions กระจาย, เพิ่ม strategy ใหม่ = แก้หลายไฟล์

หลัง **OOP (Part III)**: เพิ่ม strategy ใหม่ = สร้าง class ใหม่ที่ implement ABC

→ ทำงานกับ Backtester และ TradingSystem ทันที

- Strategy ABC บังคับ interface ✓
- Portfolio เก็บ state ครบ ✓
- EventBus แยก concerns ✓
- StrategyFactory + config สร้างระบบจาก JSON ✓
- TradingSystem ประกอบทุกอย่างผ่านจุดเดียว ✓

หมายเหตุเรื่องประสิทธิภาพ: Backtester ใน Part III ใช้ Python for-loop ซึ่งช้ากว่า vectorized ใน Part IV ประมาณ **50–100×** ใช้เพื่อเรียนรู้ logic ได้ แต่สำหรับ parameter search ให้ใช้ vectorized ใน Part IV เท่านั้น

ใน **Part IV** เราจะนำ system นี้ไป backtest อย่างจริงจัง: walk-forward, slippage, commission, overfitting